

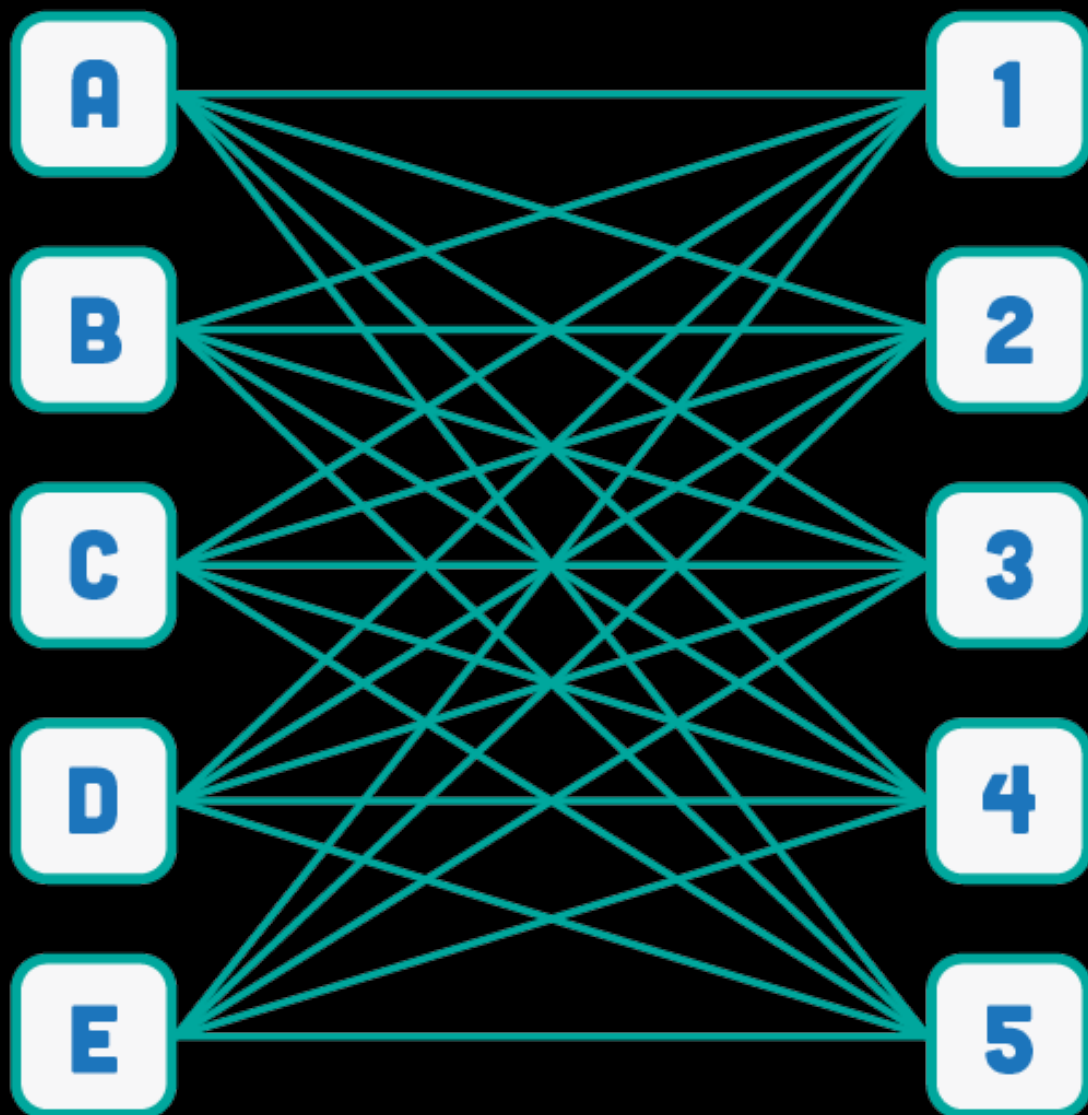
Introduction to Middleware

Service-Oriented Computing (CSC 4222)

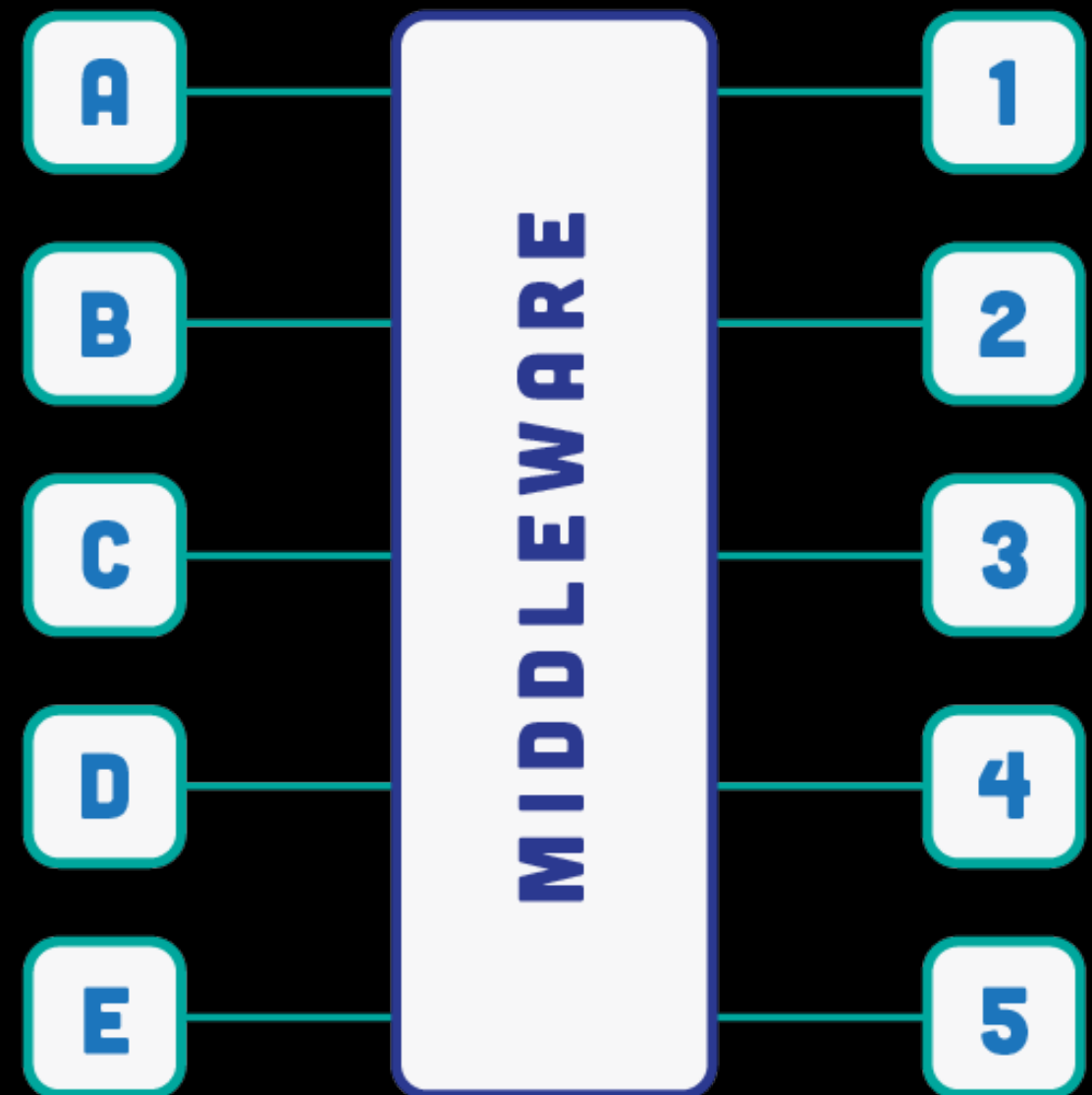
What is Middleware

- Layer between **Operating System** and Distributed **Applications**.

WITHOUT MIDDLEWARE



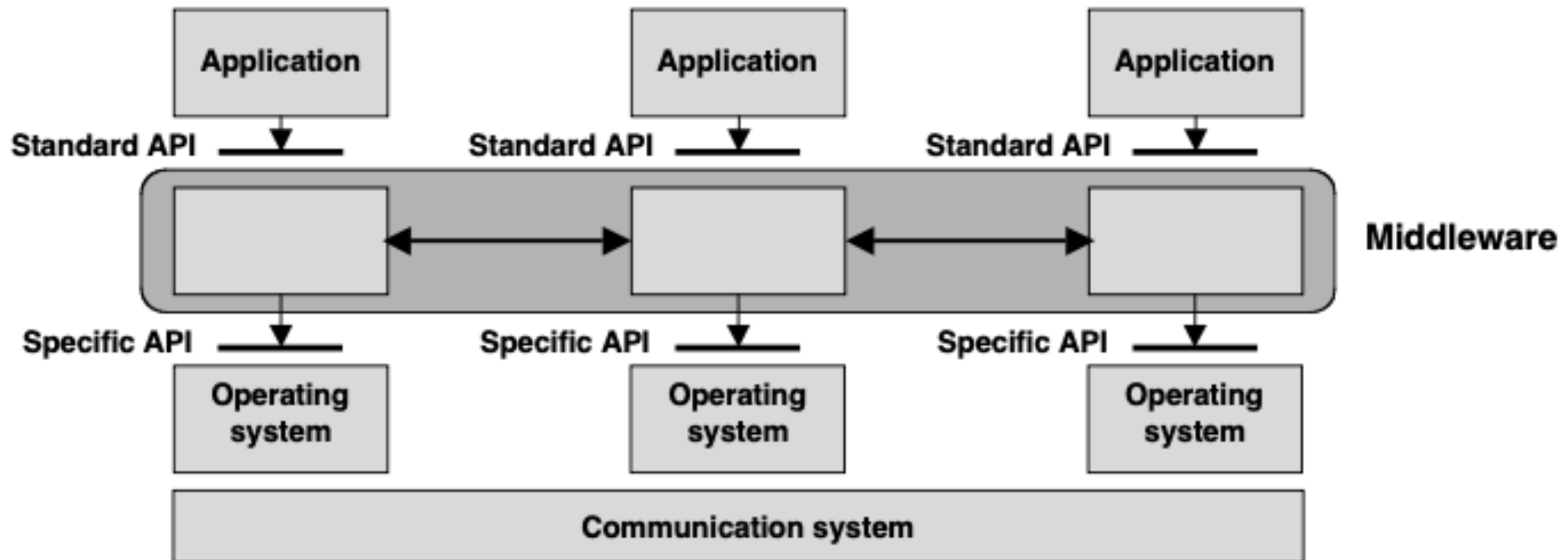
WITH MIDDLEWARE



What is Middleware ?

- **Intermediate** software layer between **applications**
- **General** purpose or **specific**, e.g. ESB, Pub-Sub
- Middleware **benefits** are mostly derived from **abstraction**:
 - **Hide** low-level **details**
 - Language and platform **independence**
 - **Reuse** of **expertise** and **code**
 - **Ease** of application **evolution**
- **Drawbacks** - Possible **performance penalty** linked to the use of multiple software layers, a significant **burden on training** app developers.

What is Middleware ?



Middleware Architecture Properties

- **Managed entities:** Objects, agents and components
- **Service provision structure:** Predefined roles of entities - client, server, publisher, subscriber.
- **Service provision interface:** Synchronous or Asynchronous

Importance of Middleware

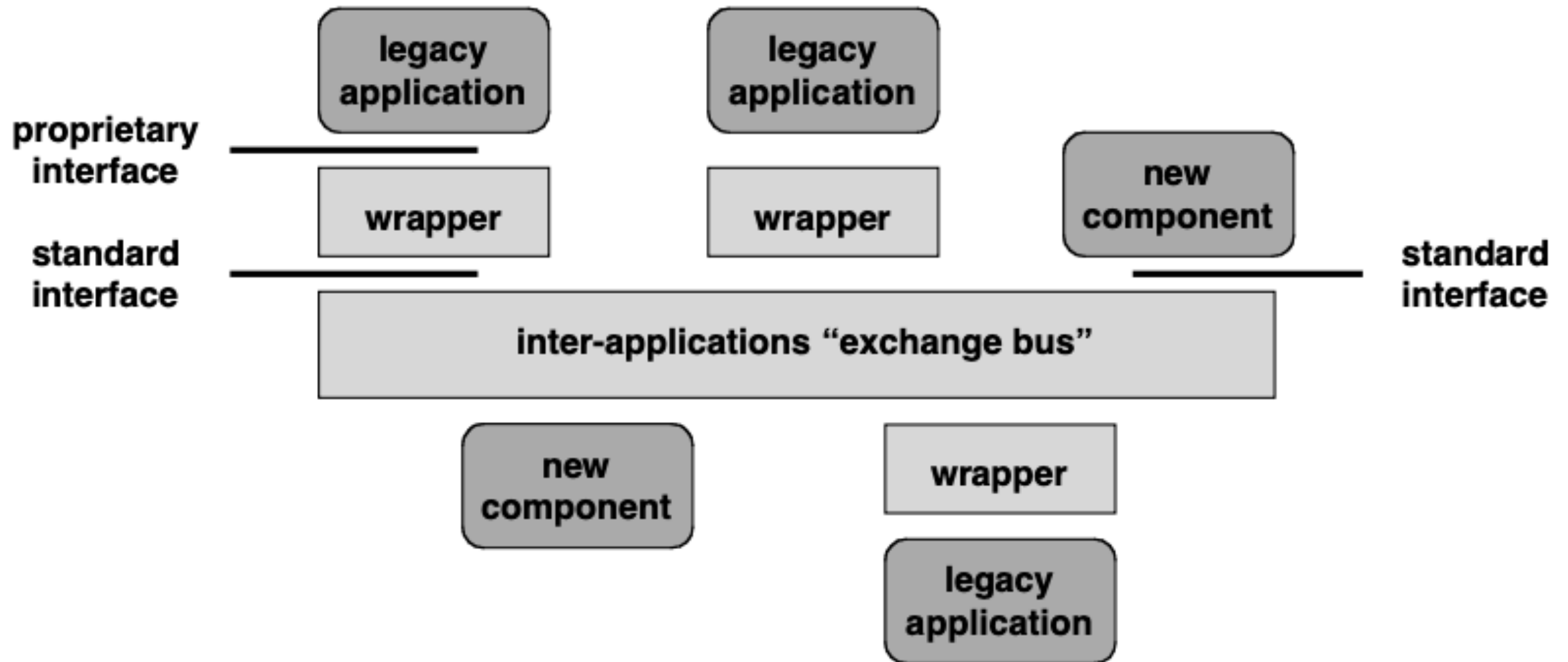
Importance of Middleware

- Reusing **Legacy** software
- As a **mediation** system - facilitates communication
- Result of **component-based** architecture
- Client adaptation through **proxies**

Reuse Legacy Software

- Legacy applications can be used through their **specific interfaces** and **cannot be modified**.
- Common **language-independent layer** to interconnect different applications - middleware/bus
- Build a “**wrapper**” - a piece of software layer that serves as a **bridge** between the legacy app interface and interconnecting middleware.
- The wrapped legacy application is **integrated** with other **interconnecting** apps connected to the **middleware/bus**.

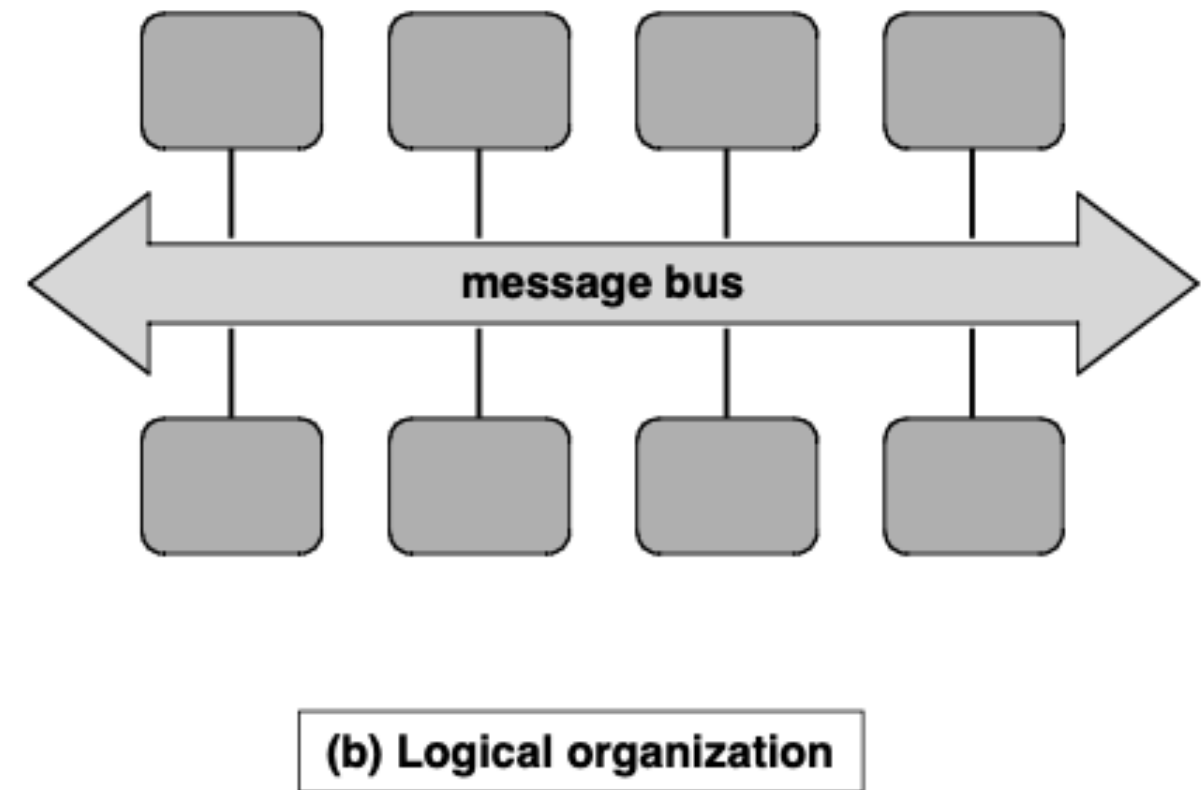
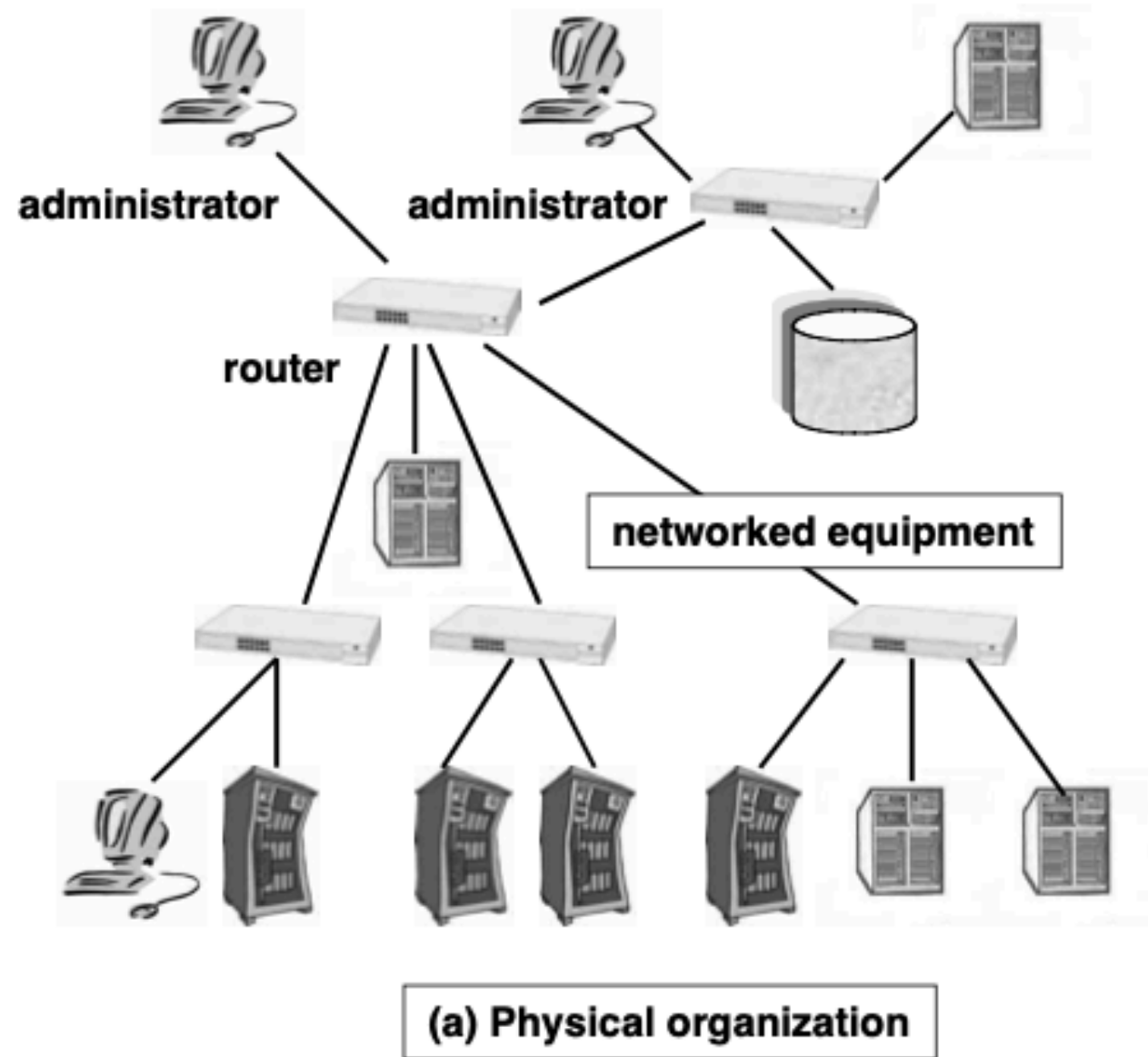
Reuse Legacy Software



As a mediation system

- **Mediation** -monitor performance, capture usage patterns, log alarms, collect billing info, execute remote maintenance functions, download and install new services.
- Interconnected distributed applications need a **common channel** to **communicate** among participants.
- Message senders and receivers - typically **asynchronous** messages
- Typical use of a message Bus - **Publish and Subscribe**.
- Members of a predefined group subscribe to a specified **“topic”**.

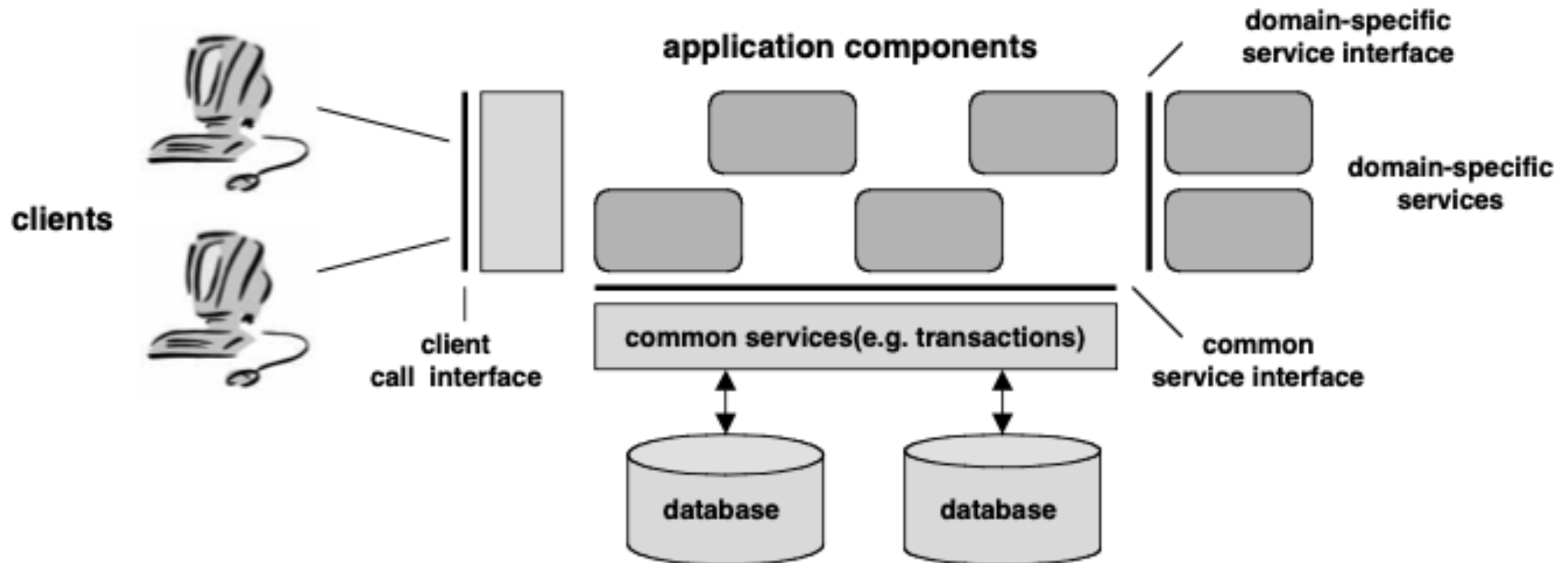
As a mediation system - facilitate communication



Component based Architectures

- Components rely on a **separation of functions** and on well-defined standard interfaces, e.g. 3-tier architecture
- Relies on support infrastructure that provides an environment for components, as well as **a common set of services** such as transaction management, and security.
- Allow developers to concentrate on **application-specific problems**.
- Improve **scalability** by separating application and database layers.

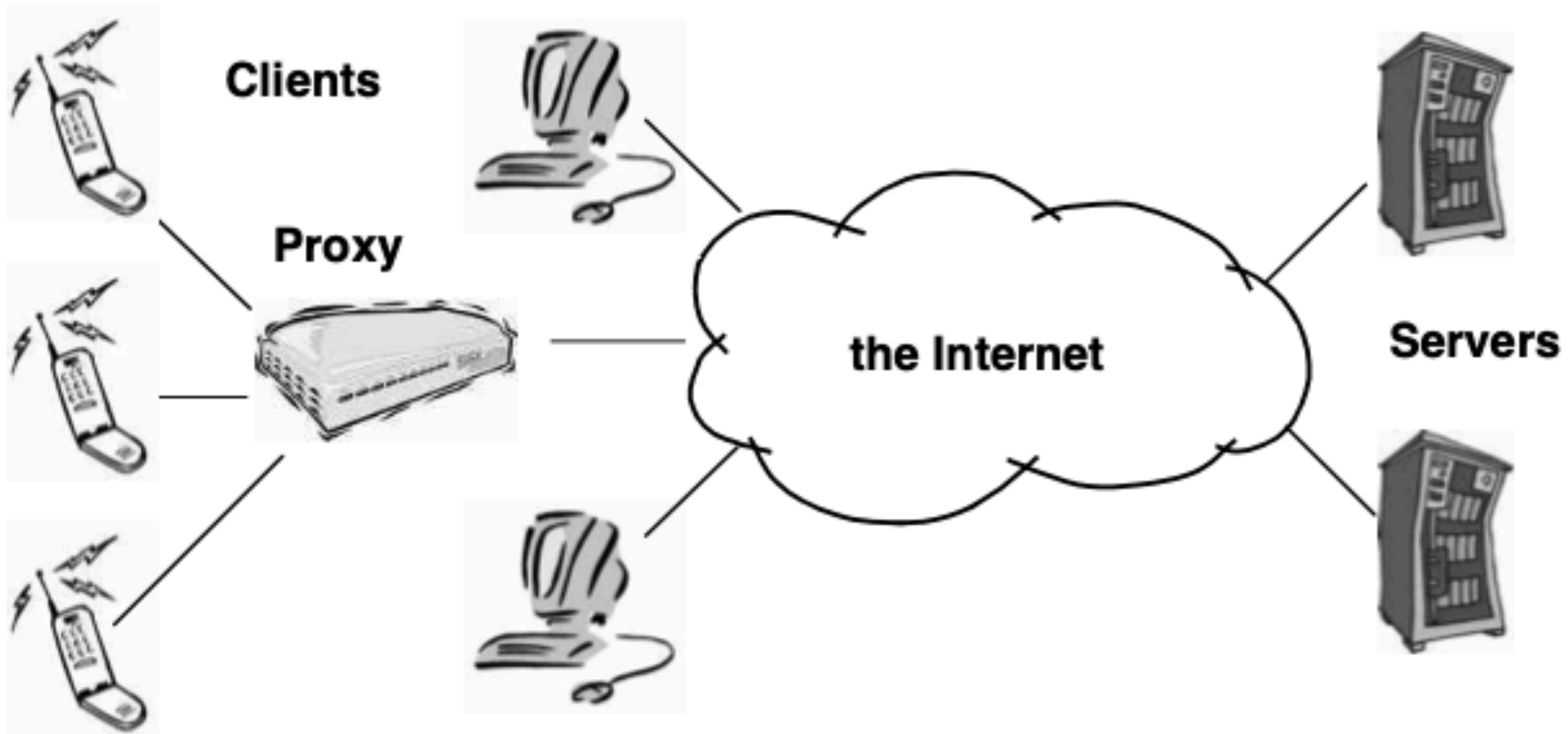
Component based Architectures



Client Adaptation through Proxies

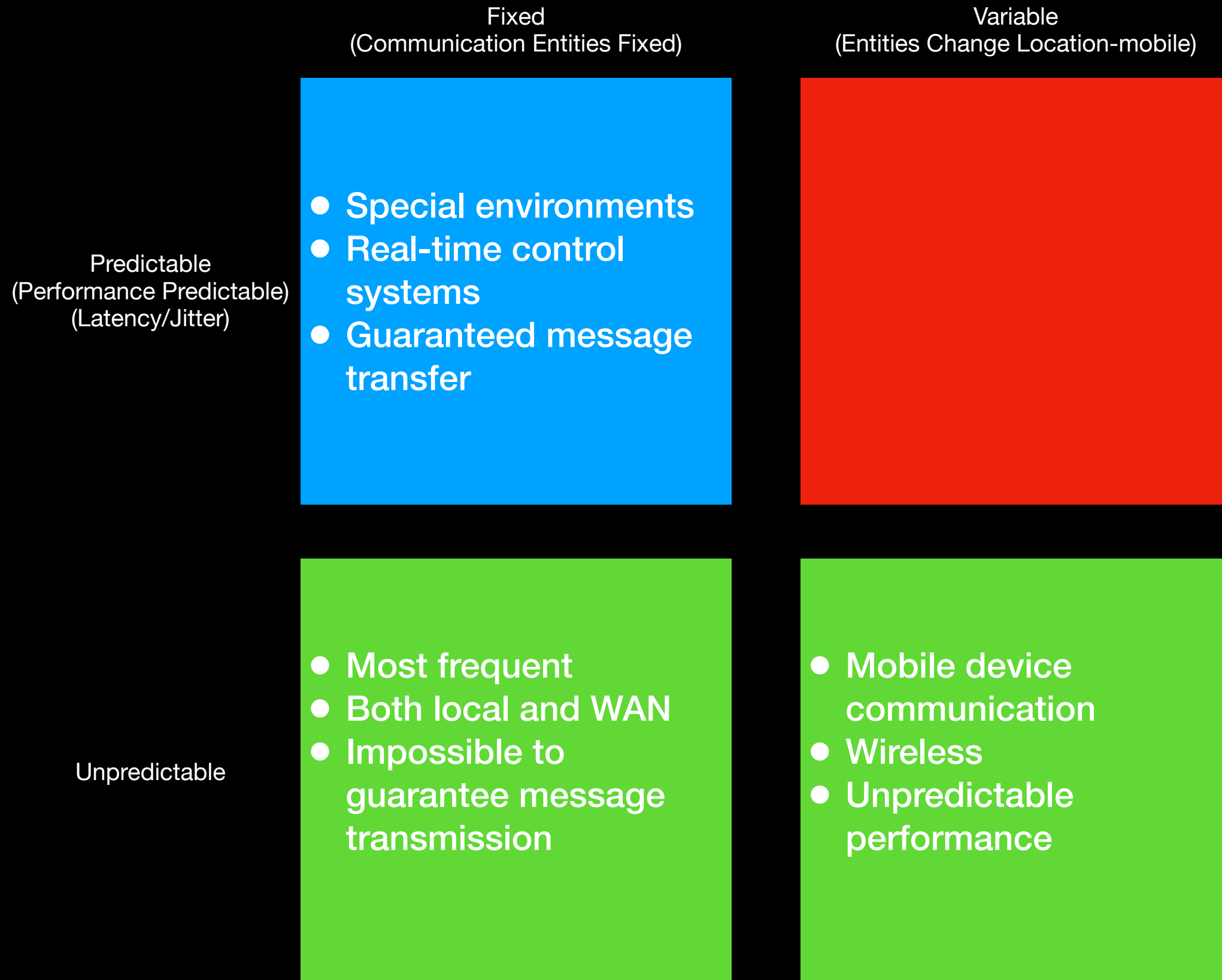
- Adaptation layer (**proxy**) between clients and servers.
- A different proxy may be designed for each **class of client devices**, e.g. phones, and desktops.
- The **function** of a proxy is to **adapt the communication flow** to and from the client.

Client Adaptation through Proxies



Categories of Middleware

Categories of Middleware

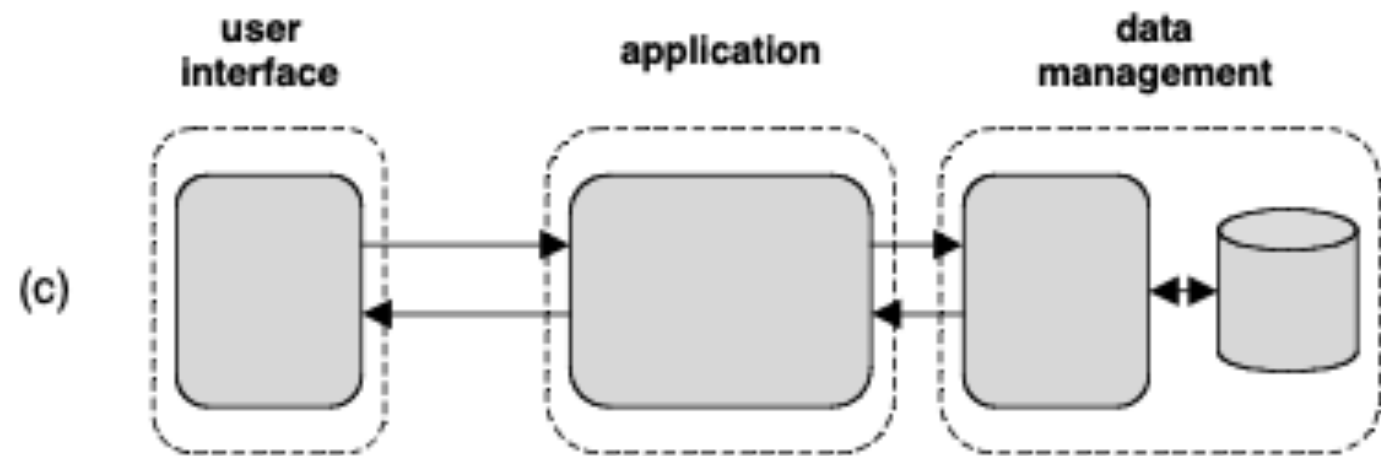
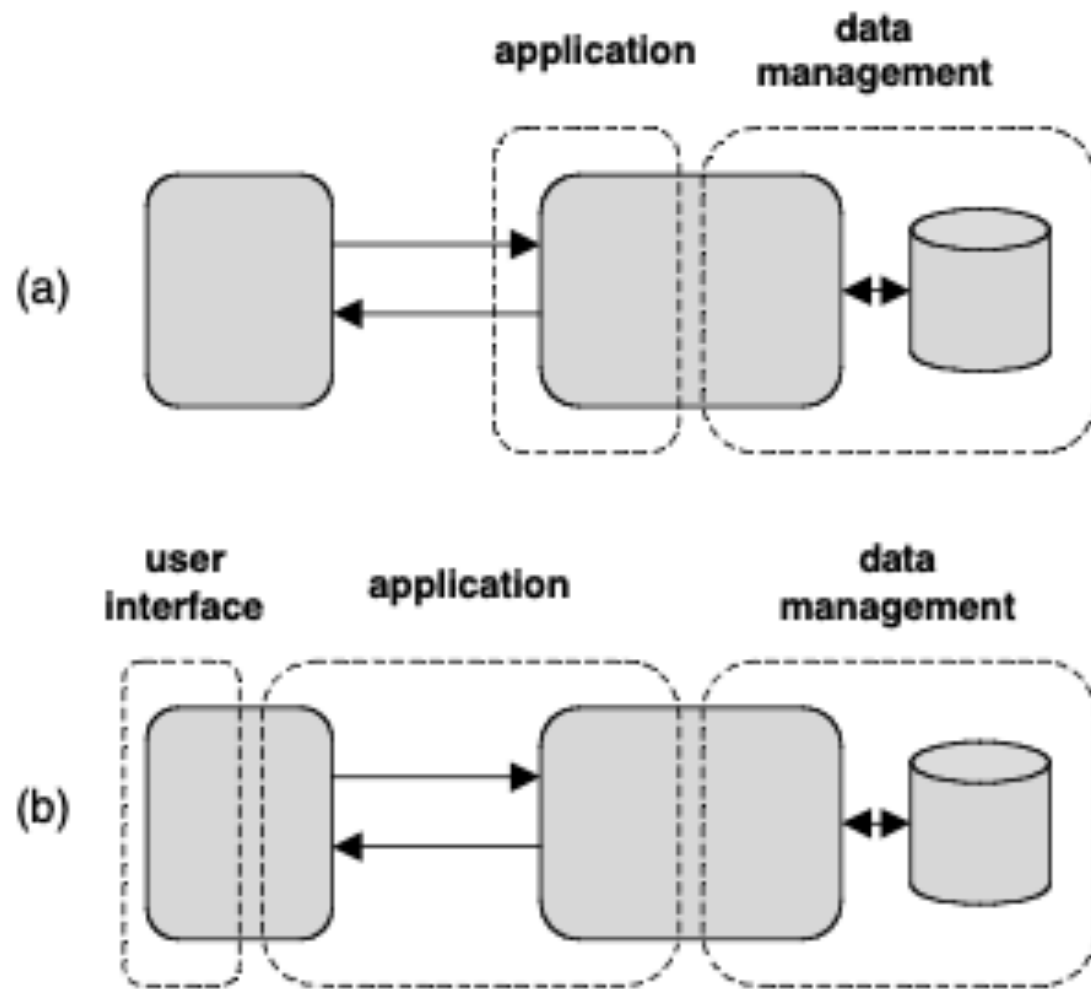


Middleware Structuring Principles

Multi-Tiered Architecture

- Overall functions of an App are segmented into several interdependent tiers.
- Tiers separate application functional domains - e.g. presentation, business logic, persistence.
- Each tier may be subject to a “vertical layered” decomposition into abstract levels - layered architecture.
- Each tier can be scaled independently to upgrade performance.

Multi-Tiered Architecture



(a) Less processing in client tier - presentation only

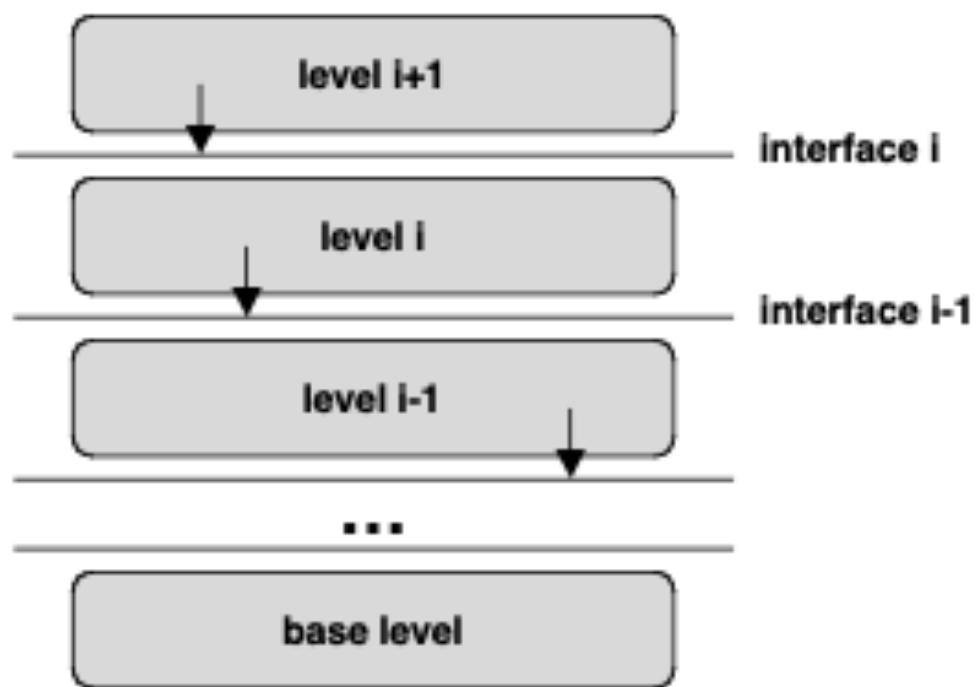
(b) Additional processing in the client tier - presentation + validation

(c) Tiered persistence - ORM + DB

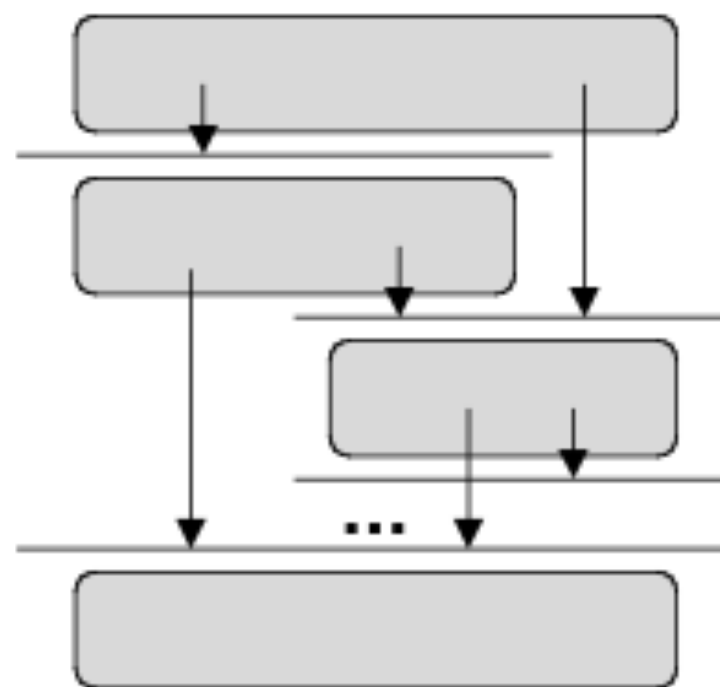
Multi-Level Layered Architecture

- Each **level** defines its own **entities/components**.
- **Layers** can be considered as **vertically organised** focusing on closely **related functions** (Single function domain) - e.g. TCP/IP, DBMS(ORM+DBMS)
- The **upper layer depends** on the **immediate lower layer**.
- The **interface** provided by **each layer** may be viewed as a set of **functions** similar to an Application Programming Interface (**API**).
- **A layer encapsulates** the implementation and only offers an **interface** for an **upper layer** to use.
- E.g. JVM

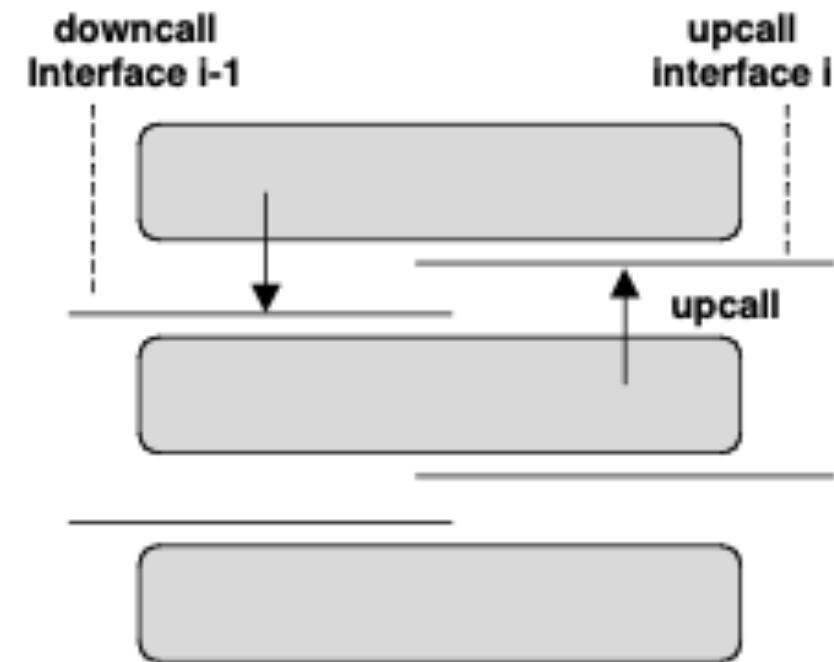
Multi-Level Layered Architecture



Fully Dependent



Partially Dependent

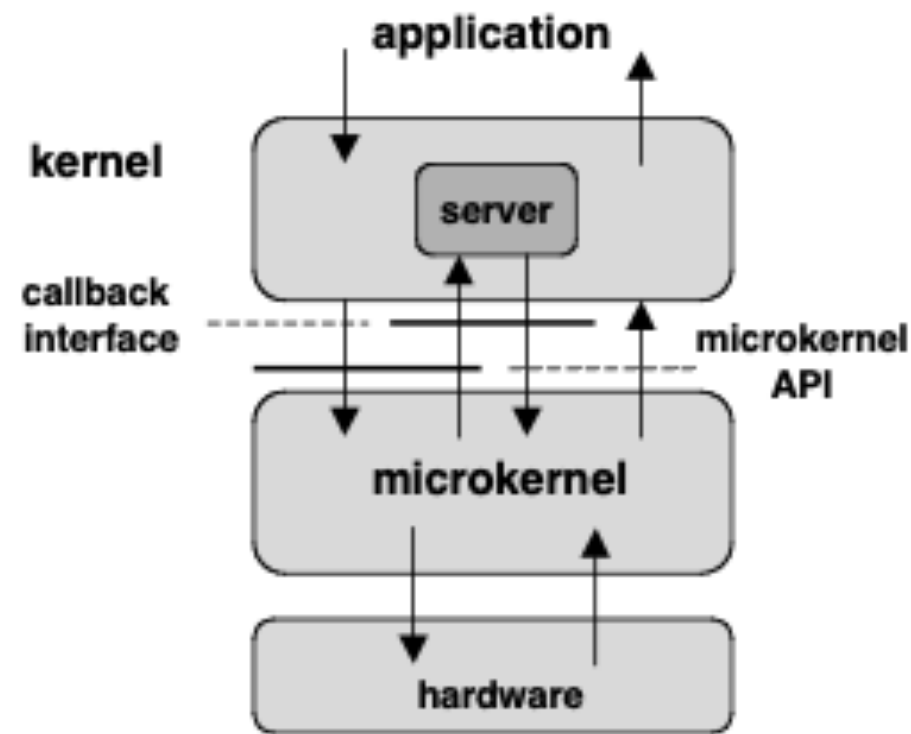


**Fully Dependent with
up-calls(Async)**

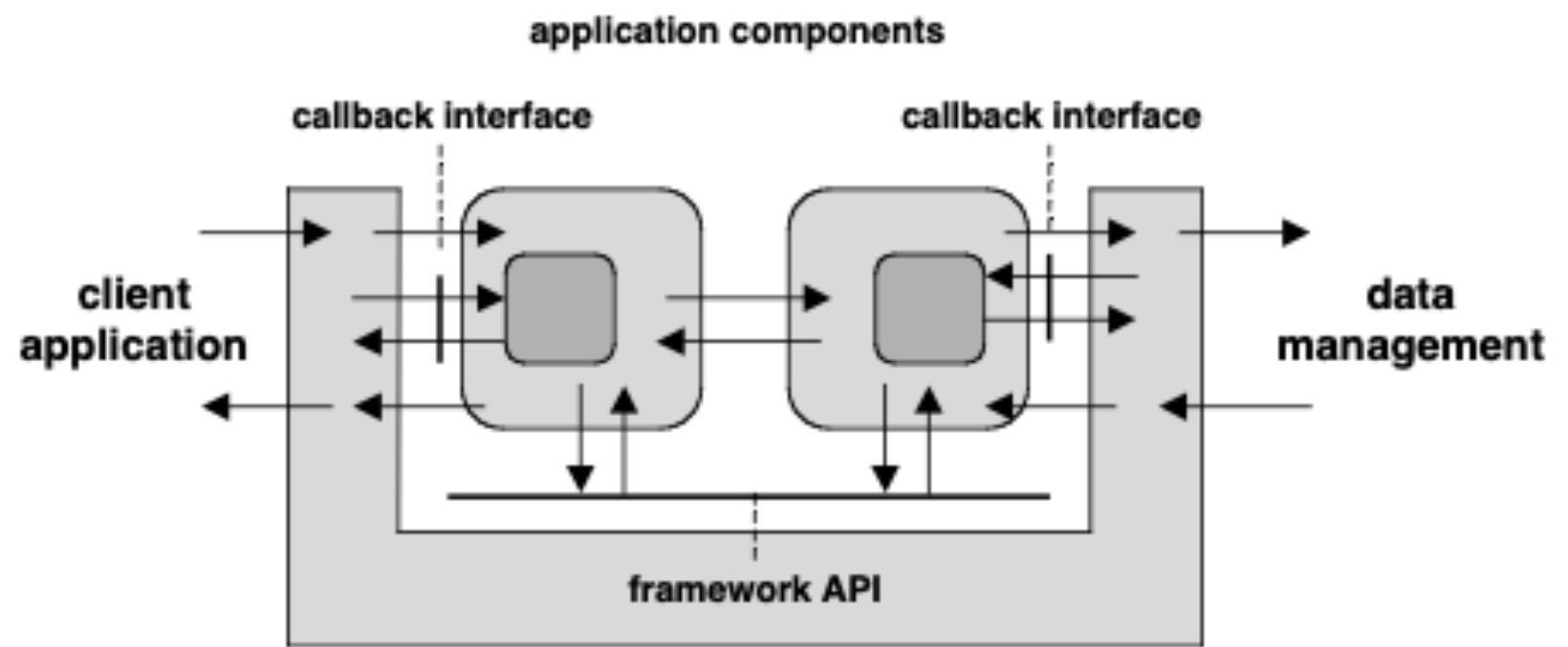
Frameworks

- Program skeleton that may be directly reused to solve a family of related problems.
- Components could be added to a framework aligned with the rules to implement application functions.
- e.g. Inherit from a class to capture HTTP requests, and implement a framework interface to enable call-back functionality.
- E.g. Application frameworks - Springboot, .NET

Frameworks



(a) microkernel

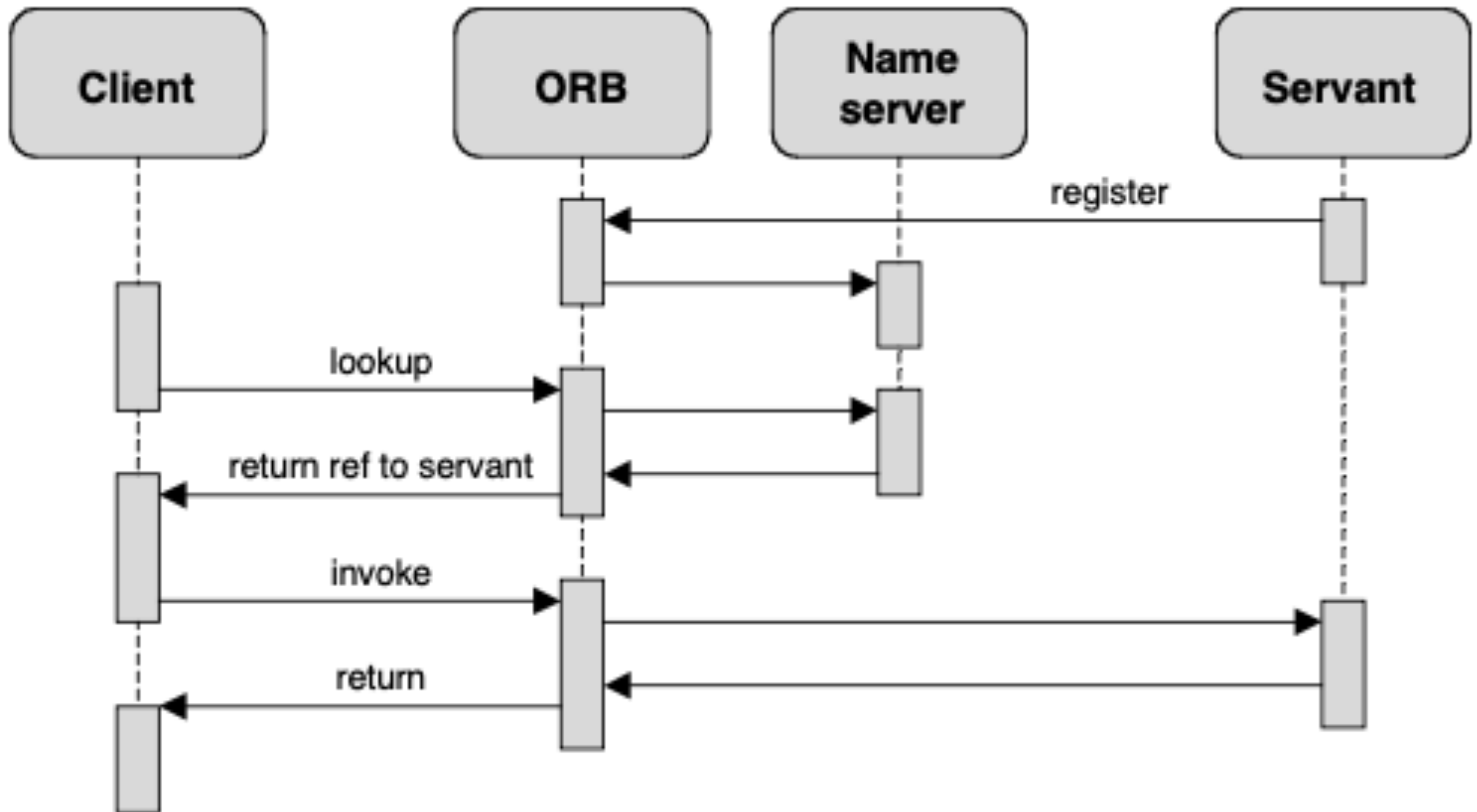


(b) middle tier framework

Distributed Objects

- Objects define **interfaces** to be called in a **distributed environment**.
- Objects could be on **heterogeneous** platforms and languages - C++, Java, Unix, Windows
- Dynamic remote **object creation** - RMI, CORBA
- Implement new objects by **inheriting** from **generic base classes** to implement new functions.

Distributed Objects



E.g. Remote Method Invocation (RMI)

{ e n d }

Be the change you wish to see in the world

Mahatma Gandhi